Representation Alignment for Generation:
Training Diffusion Transformers Is Easier Than You Think

Sihyun Yu[1] Sangkyung Kwak[1] Huiwon Jang[1] Jongheon Jeong[2]
Jonathan Huang[3] Jinwoo Shin[1*] Saining Xie[4*]

[1]KAIST [2]Korea University [3]Scaled Foundations [4]New York University
[*]Equal Advising

ICLR 2025 (Oral)

Outline

1. Introduction & Motivation
2. Key Observations
3. Method: REPA
4. Bridging the Representation Gap
5. Experimental Results
6. Conclusion

Motivation

Diffusion Transformers (DiT, SiT) are the new paradigm for image generation:

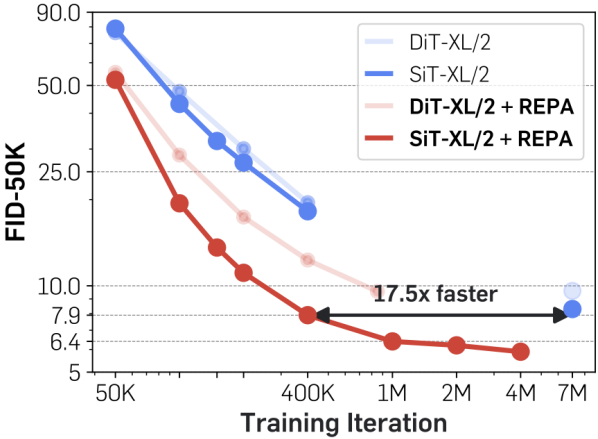
- State-of-the-art on ImageNet generation
- Scale well with model size and compute

Key challenge:

- Training is extremely expensive (SiT-XL/2: **7M steps**)
- Bottleneck: Learning good internal representations

This paper's insight:

- Use **pretrained visual encoders** to guide representation learning
- Simple regularization $\Rightarrow$ **17.5$\times$ faster** training



Background: Flow Matching & Diffusion Transformers

Flow Matching / Interpolant Framework:
Given clean data x_0 and noise $\epsilon \sim \mathcal{N}(0, I)$, construct noisy input:

$$x_t = \alpha_t x_0 + \sigma_t \epsilon \tag{1}$$

Linear path: $\alpha_t = 1 - t, \quad \sigma_t = t, \quad t \in [0, 1]$

Velocity (v-prediction) target:

$$v_t = \dot{\alpha}_t x_0 + \dot{\sigma}_t \epsilon \tag{2}$$

For linear path: $v_t = -x_0 + \epsilon$

Diffusion Transformers:

- **DiT** (Peebles & Xie, 2023): Transformer backbone replaces U-Net
- **SiT** (Ma et al., 2024): DiT + Scalable Interpolant framework
- Patch embedding $\rightarrow$ Transformer blocks with adaLN-Zero $\rightarrow$ Unpatchify

Background: Self-Supervised Visual Encoders

Contrastive / Self-distillation:

- **DINOv2** – Self-distillation with ViT
- **MoCo v3** – Momentum contrastive
- **CLIP** – Vision-language contrastive

Masked prediction:

- **MAE** – Masked autoencoder
- **I-JEPA** – Joint-embedding predictive

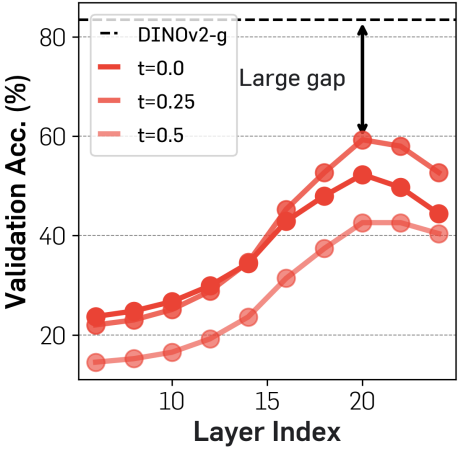
Key question: *Can we leverage these high-quality representations to help train diffusion models?*

Observation 1: Diffusion Models Learn Representations

Layer-wise linear probing on a pretrained SiT-XL/2 (7M steps):

- SiT *does* learn meaningful discriminative features
- But quality significantly lags behind DINOv2
- Middle-to-later layers show best probing accuracy

Takeaway: There is a large representation gap between diffusion models and SSL encoders.

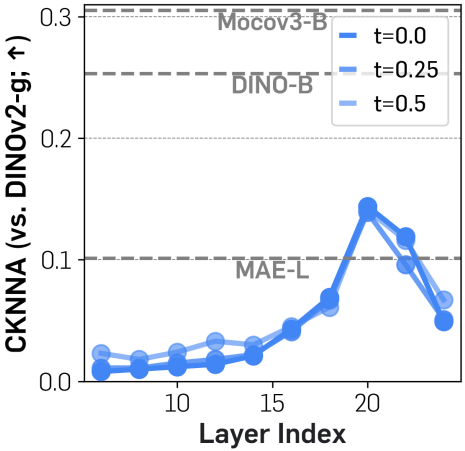


Observation 2: Weak Alignment with SSL Representations

Centered Kernel Alignment (CKA) between SiT and DINOv2:

- Measures similarity between two sets of representations
- Alignment is weak across all layers
- Early layers: almost no alignment
- Later layers: moderate but still far from perfect

Despite learning useful features, SiT representations are poorly aligned with DINOv2.

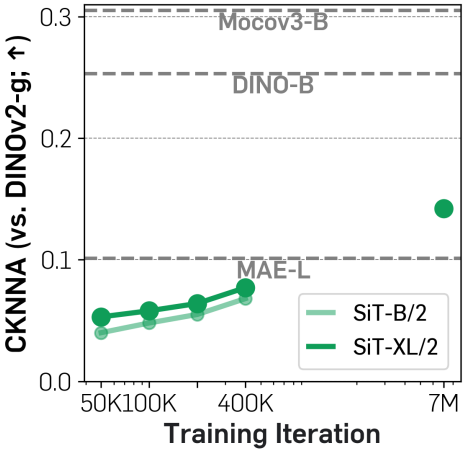


Observation 3: Alignment Improves with Longer Training

CKA alignment between SiT and DINOv2 at different training stages:

- Alignment **consistently improves** with more training steps
- Larger models align faster
- This suggests: *alignment is a natural tendency, just very slow*

Hypothesis: If we explicitly encourage this alignment, training will converge much faster!



Summary of Observations

Obs 1: Diffusion models learn discriminative representations, but they are inferior to SSL encoders (DINOv2)

↓

Obs 2: The alignment between diffusion model and DINOv2 representations is weak

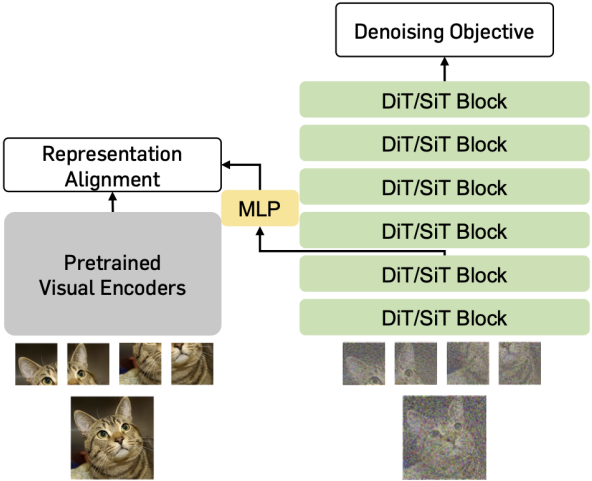
↓

Obs 3: This alignment naturally improves with longer training and larger models

↓

Idea: *Explicitly* align diffusion model hidden states with SSL representations $\Rightarrow$ **REPA**

REPA: REPresentation Alignment



Core idea:

Align the hidden representation of the denoising network (at an intermediate layer l) with the clean image representation from a frozen pretrained encoder.

Key components:

- Frozen SSL encoder (DINOv2)
- Projector MLP on hidden states
- Cosine similarity alignment loss
- No overhead at inference

Flow Matching Formulation

Stochastic interpolant:

$$x_t = \alpha_t x_0 + \sigma_t \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad t \in [0, 1] \tag{3}$$

Path types:

- **Linear:** $\alpha_t = 1 - t, \quad \sigma_t = t$
- **Cosine:** $\alpha_t = \cos\left(\frac{\pi t}{2}\right), \quad \sigma_t = \sin\left(\frac{\pi t}{2}\right)$

Velocity field (v-prediction):

$$v_t = \dot{\alpha}_t x_0 + \dot{\sigma}_t \epsilon \tag{4}$$

The model $v_\theta(x_t, t)$ is trained to predict this velocity.

Denoising loss:

$$\mathcal{L}_{\text{denoise}} = \mathbb{E}_{t, x_0, \epsilon}\left[\left\| v_\theta(x_t, t) - v_t \right\|^2\right] \tag{5}$$

REPA Alignment Loss

Setup: Given a frozen pretrained encoder f_{enc} (e.g., DINOv2):

1. Extract **clean image features**: $z = f_{\text{enc}}(x_0) \in \mathbb{R}^{N \times d_z}$ (patch tokens)
2. Extract **noisy hidden states** at layer l: $h^{(l)} = \text{DiT-Block}_l(x_t, t, y)$
3. Project hidden states: $\tilde{z} = f_{\text{proj}}(h^{(l)}) \in \mathbb{R}^{N \times d_z}$

Alignment loss (negative cosine similarity):

$$\mathcal{L}_{\text{align}} = -\mathbb{E}_{t, x_0, \epsilon} \left[\frac{1}{N} \sum_{j=1}^{N} \left\langle \frac{\tilde{z}_j}{\|\tilde{z}_j\|}, \frac{z_j}{\|z_j\|} \right\rangle \right] \tag{6}$$

where N is the number of patch tokens, and the sum is over **spatial positions**.

Intuition: Force the diffusion model's intermediate representation to "see" what DINOv2 sees in the clean image, even from noisy input.

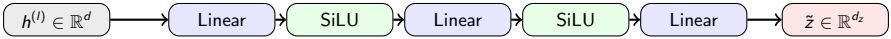
Total Training Objective

Combined loss:

$$\boxed{\mathcal{L} = \mathcal{L}_{\text{denoise}} + \lambda \cdot \mathcal{L}_{\text{align}}} \tag{7}$$

where λ controls the alignment strength (default: $\lambda = 0.5$).

Projector MLP f_{proj}: maps hidden states to encoder space



Dimensions: $d \to 2048 \to 2048 \to d_z$ (projector_dim $= 2048$)

Design Choices

Which layer to align?

- Alignment applied at layer $l = \texttt{encoder_depth}$
- Default: $l = 8$ for SiT-XL/2 (28 total blocks)
- **Early layers** $\Rightarrow$ aligns low-level + semantic features

Which encoder?

- **DINOv2** (ViT-B, ViT-L, ViT-g) – best overall
- Also works with: CLIP, MoCo v3, I-JEPA, MAE, DINOv1

Key design principle:

Align **early layers** with SSL encoder
$\Rightarrow$ Later layers are freed to focus on **high-frequency generation details**

Other details:

- Projector is *only used during training* (discarded at inference)
- Encoder is *frozen* – no gradient flows back
- $\lambda = 0.5$, AdamW optimizer, lr $= 10^{-4}$, batch size $= 256$

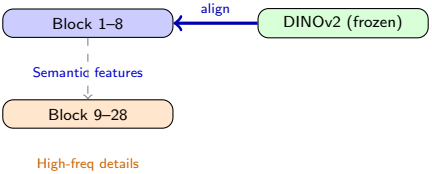
How REPA Works: Layer Specialization

Without REPA (Vanilla):

- All layers must jointly learn both *semantic understanding* and *fine-grained generation*
- Slow convergence – early layers struggle

With REPA:

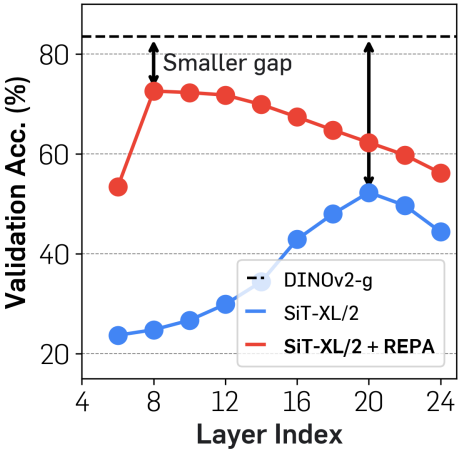
- **Early layers** ($\leq l$): Aligned with SSL encoder $\Rightarrow$ semantics "injected"
- **Later layers** ($> l$): Free for high-freq details



REPA Bridges the Representation Gap: Linear Evaluation

Linear probing accuracy per layer:

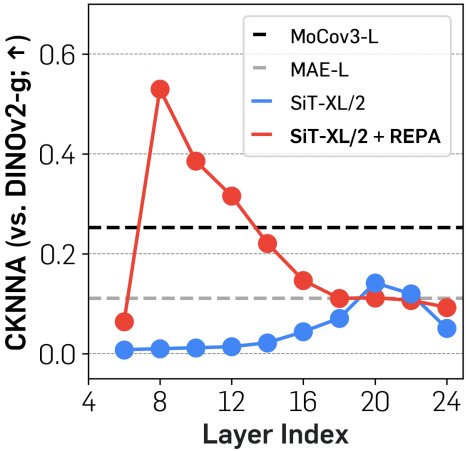
- REPA dramatically improves representation quality at **early layers**
- Representations become comparable to DINOv2 in quality
- This confirms: alignment is *successfully transferred*



REPA Bridges the Representation Gap: CKA Alignment

CKA alignment with DINOv2:

- REPA achieves **much stronger** alignment, especially at early layers
- Alignment at layer 8 (the target) is very high
- Later layers remain *different* – they specialize in generation

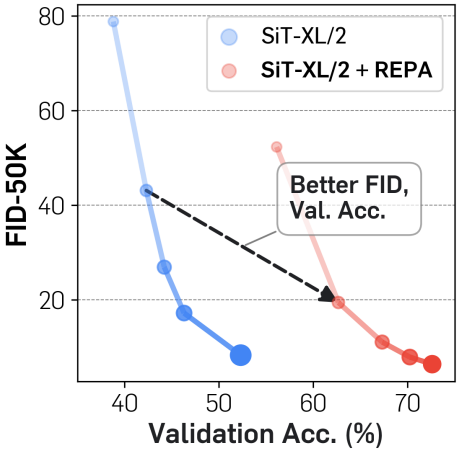


Layer Specialization: Representation Slope

Representation slope measures how much each layer changes the representation:

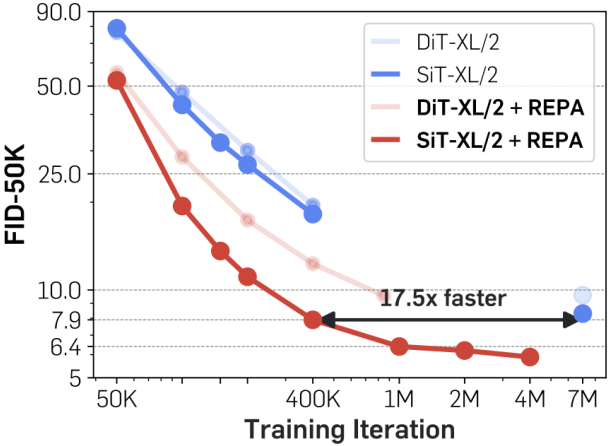
- With REPA: a **clear transition** at layer l
- Layers $\leq l$: smooth, aligned representations
- Layers $> l$: sharp changes for generation-specific processing

Conclusion: REPA creates a natural **division of labor** between semantic encoding and generative decoding.

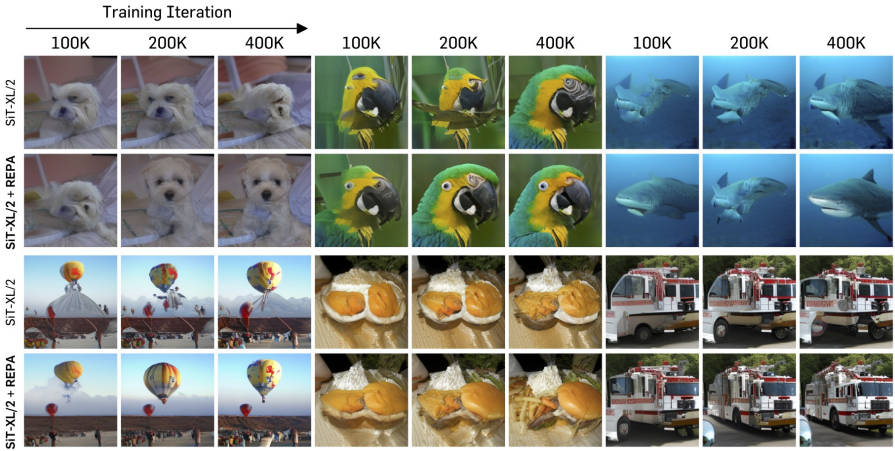


Result 1: 17.5× Faster Training

- REPA matches vanilla SiT-XL/2 (7M steps) in <**400K steps**
- **17.5× speedup** in training convergence
- Consistent improvement across FID, IS, and linear probing metrics



Result 2: Qualitative Progression During Training



Top: Vanilla SiT-XL/2 **Bottom:** SiT-XL/2 + REPA

Same noise, same sampler, same steps. REPA produces recognizable images much earlier.

Result 3: Generation Quality without CFG

- Without CFG, REPA achieves **FID = 7.9** at only 400K steps
- Outperforms vanilla SiT-XL/2 trained for **7M steps** (FID = 9.9)
- Also improves Inception Score and linear probing accuracy

Model	#Params	Iter.	FID↓
DiT-L/2	458M	400K	23.3
+ REPA (ours)	458M	**400K**	**15.6**
DiT-XL/2	675M	400K	19.5
+ REPA (ours)	675M	**400K**	**12.3**
DiT-XL/2	675M	**7M**	9.6
+ REPA (ours)	675M	**850K**	**9.6**
SiT-B/2	130M	400K	33.0
+ REPA (ours)	130M	**400K**	**24.4**
SiT-L/2	458M	400K	18.8
+ REPA (ours)	458M	**400K**	**9.7**
+ REPA (ours)	458M	**700K**	**8.4**
SiT-XL/2	675M	7M	8.3
+ REPA (ours)	675M	**400K**	**7.9**
+ REPA (ours)	675M	**1M**	**6.4**
+ REPA (ours)	675M	**4M**	**5.9**

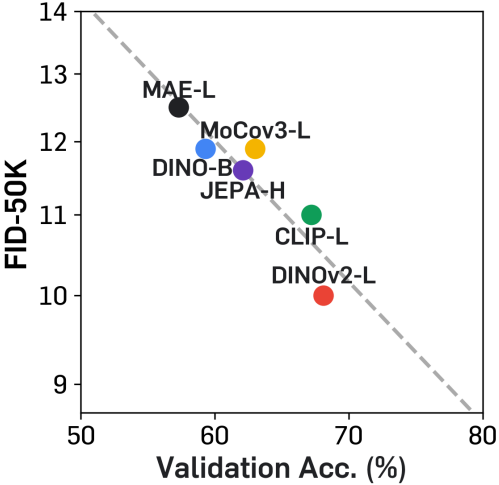
Result 4: State-of-the-Art with CFG

- With CFG + guidance interval, REPA achieves **FID = 1.42** (state-of-the-art)
- Outperforms all previous diffusion models with **7× fewer epochs**
- Works with both SDE and ODE sampling

Model	Epochs	FID↓	sFID↓	IS↑	Pre.↑	Rec.↑
Pixel diffusion						
ADM-U	400	3.94	6.14	186.7	0.82	0.52
VDM++	560	2.40	-	225.3	-	-
Simple diffusion	800	2.77	-	211.8	-	-
CDM	2160	4.88	-	158.7	-	-
Latent diffusion, U-Net						
LDM-4	200	3.60	-	247.7	0.87	0.48
Latent diffusion, Transformer + U-Net hybrid						
U-ViT-H/2	240	2.29	5.68	263.9	0.82	0.57
DiffiT*	-	1.73	-	276.5	0.80	0.62
MDTv2-XL/2*	1080	1.58	4.52	314.7	0.79	0.65
Latent diffusion, Transformer						
MaskDiT	1600	2.28	5.67	276.6	0.80	0.61
SD-DiT	480	3.23	-	-	-	-
DiT-XL/2	1400	2.27	4.60	278.2	**0.83**	0.57
SiT-XL/2	1400	2.06	4.50	270.3	0.82	0.59
+ REPA (ours)	200	1.96	**4.49**	264.0	0.82	0.60
+ REPA (ours)	800	1.80	4.50	284.0	0.81	0.61
+ REPA (ours)*	**800**	**1.42**	4.70	**305.7**	0.80	**0.65**

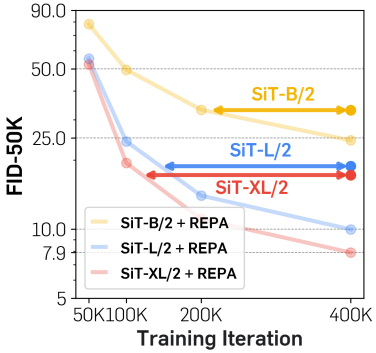
Result 5: Scalability Across Encoders

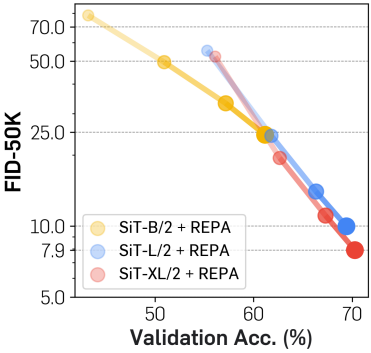
- Better encoder ⇒ better generation **and** representation
- **DINOv2** family performs best overall
- Even weaker encoders (MAE, DINOv1) improve over vanilla



Result 6: Scaling Laws

- REPA provides **greater speedup for larger models**
- **Log-linear correlation** between linear probing accuracy and FID
- Improving representations directly translates to better generation





Qualitative Samples (SiT-XL/2 + REPA)

Key Takeaways

1. **Representation quality is the bottleneck** in training diffusion transformers
2. **REPA** is a simple regularization: align diffusion model hidden states with pretrained SSL encoders using cosine similarity
3. Results are **striking**:
   - **17.5**$\times$ training speedup
   - State-of-the-art **FID = 1.42** on ImageNet 256$\times$256
   - Works with diverse encoders (DINOv2, CLIP, I-JEPA, MAE, ...)
4. REPA creates **layer specialization**: early layers for semantics, later layers for generation details
5. **Practical takeaway:** Training powerful diffusion transformers is now *much easier and cheaper*

Limitations & Future Work

Limitations:

- Requires access to a pretrained visual encoder (additional dependency)
- Currently demonstrated primarily on ImageNet – scaling to diverse domains needs study
- The projector MLP adds a small overhead during training (discarded at inference)

Future directions:

- Apply REPA to **text-to-image** generation at scale (initial MS-COCO results shown)
- Explore alignment with **multi-modal encoders** (e.g., CLIP for text-guided alignment)
- Combine with **progressive training** or **distillation** for even faster convergence
- Investigate **adaptive alignment depth** – dynamically choose layer l

Thank for your listening!

Paper: https://arxiv.org/abs/2410.06940
Code: https://github.com/sihyun-yu/REPA
Project: https://sihyun.me/REPA/

Appendix: Training Hyperparameters

Hyperparameter	Value
Optimizer	AdamW
Learning rate	10^{-4} (constant)
Betas	(0.9, 0.999)
Weight decay	0
Batch size	256
Max steps	400K / 7M
Mixed precision	FP16
EMA decay	0.9999
Grad clipping	norm ≤ 1.0
Path type	Linear
Prediction	v-prediction
Weighting	Uniform
CFG dropout	0.1

REPA-specific:

Parameter	Value
λ (align coeff.)	0.5
Encoder depth l	8
Projector dim	2048
Encoder	DINOv2-ViT-B

Notes:

- Projector discarded at inference
- Encoder frozen throughout
- EMA for final evaluation

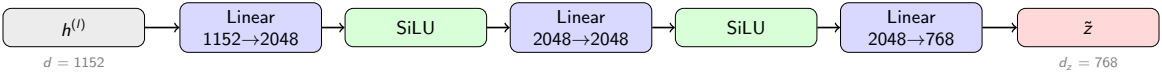
Appendix: Model Configurations

Model	Depth	Hidden	Heads	Patch	Params
SiT-B/2	12	768	12	2	∼130M
SiT-L/2	24	1024	16	2	∼458M
SiT-XL/2	28	1152	16	2	∼675M
Pretrained Encoders (frozen):					
DINOv2-ViT-B	12	768	12	14	∼86M
DINOv2-ViT-L	24	1024	16	14	∼304M
DINOv2-ViT-g	40	1536	24	14	∼1.1B

Resolution: 256×256 images, encoded to 32×32 latent (VAE)
Latent channels: 4 (SD-VAE-ft-mse)
Number of patches: $32/2 = 16 \Rightarrow 16 \times 16 = 256$ patch tokens

Appendix: Projector MLP Architecture

3-layer MLP with SiLU activation (used during training only):



Key properties:

- Trainable parameters: ∼8M extra
- **Discarded at inference** – no speed penalty
- Applied only to the first l layers' output

Normalization:

- Both $\tilde{z}$ and z are L2-normalized before cosine similarity
- Scale-invariant loss, direction only

Ablation: Which Layer to Align? (Encoder Depth l)

Encoder Depth l	**FID-50K ↓**	**IS ↑**	**Linear Acc. ↑**	**Notes**
Vanilla (no REPA)	52.3	—	40.2	Baseline
$l = 2$	36.4	—	55.8	Too shallow
$l = 4$	28.1	—	62.3	Better but not optimal
$l = 8$	**23.8**	—	**66.1**	**Best (default)**
$l = 12$	26.5	—	63.7	Slightly worse
$l = 16$	31.2	—	59.4	Too deep

Key findings:

- **Layer 8** is the sweet spot for SiT-XL/2 (28 blocks)
- Too shallow ($l = 2$): not enough capacity before alignment point
- Too deep ($l = 16$): later layers don't have enough room for generation-specific processing
- Rule of thumb: align at ∼25–30% of total depth

Ablation: Alignment Coefficient λ

λ	**FID-50K** $\downarrow$	**Linear Acc.** $\uparrow$	**Notes**
$\lambda = 0$	52.3	40.2	Vanilla (no REPA)
$\lambda = 0.1$	32.7	58.3	Mild alignment
$\lambda = 0.25$	26.1	63.5	Good
$\lambda = 0.5$	**23.8**	**66.1**	**Default**
$\lambda = 1.0$	24.5	65.8	Slightly aggressive
$\lambda = 2.0$	27.3	64.2	Too strong

Key findings:

- $\lambda = 0.5$ provides the best balance between denoising and alignment
- Even small λ gives a significant boost over vanilla
- Too large λ overweights alignment $\Rightarrow$ hurts generation quality
- REPA is **robust** across a wide range ($\lambda \in [0.25, 1.0]$)

Ablation: Loss Function Variants

What type of alignment loss works best?

Loss Type	Formula	FID ↓	Linear Acc.
Cosine sim. (REPA)	$-\langle \bar{\tilde{z}}, \bar{z} \rangle$	**23.8**	**66.1**
MSE	$\|\tilde{z} - z\|^2$	28.4	62.7
Smooth L1	smooth-$\ell_1(\tilde{z}, z)$	27.9	63.1

Why cosine similarity?

- Normalization makes the loss **scale-invariant**
- Focuses on **directional alignment**, not magnitude
- More stable training – avoids magnitude mismatch between different encoder spaces
- Consistent with self-supervised learning objectives (e.g., DINO, BYOL)

Appendix: Extension to Text-to-Image Generation

REPA also works for text-to-image generation (MS-COCO):

Setup:

- Model: U-ViT architecture
- Encoder: DINOv2-ViT-B (frozen)
- Text conditioning: CLIP text encoder
- Dataset: MS-COCO (256×256)

Result:

- Significant FID improvement over vanilla U-ViT
- Faster convergence (similar to class-conditional setting)
- REPA is **architecture-agnostic** and **task-agnostic**

Key insight:

The representation alignment principle is general:

Any denoising model benefits from aligning its internal features with high-quality external representations, regardless of conditioning type.

Appendix: Higher Resolution (512×512)

REPA scales to higher resolution:

Model	Resolution	FID-50K ↓	Steps
SiT-XL/2 (vanilla)	256×256	9.9	7M
SiT-XL/2 + REPA	256×256	**7.9**	400K
SiT-XL/2 + REPA	512×512	—	400K

Implementation details for 512×512:

- Images resized to 512×512, then encoded to 64×64 latent via SD-VAE
- Number of patches: $64/2 = 32 \Rightarrow 32 \times 32 = 1024$ tokens
- DINOv2 input: interpolated to $224 \times (512/256) = 448$ resolution
- Same REPA hyperparameters ($\lambda = 0.5$, $l = 8$) transfer well

Appendix: Sampling Methods & Guidance

REPA works with different samplers and guidance strategies:

Sampler	Steps	CFG Scale	Guidance Interval	FID ↓
Without CFG:				
Euler ODE	250	1.0	—	7.9
With CFG:				
Euler ODE	250	1.5	$[0, 1]$	2.15
SDE	250	1.8	$[0, 0.7]$	**1.42**
Heun ODE	250	1.5	$[0, 1]$	1.80

Guidance interval (Kynkäänniemi et al., 2024):

- Apply CFG only during a specific noise range $[t_{\text{low}}, t_{\text{high}}]$
- SDE sampler + guidance interval $[0, 0.7]$ achieves **FID = 1.42**
- Reduces artifacts from over-guidance at low noise levels